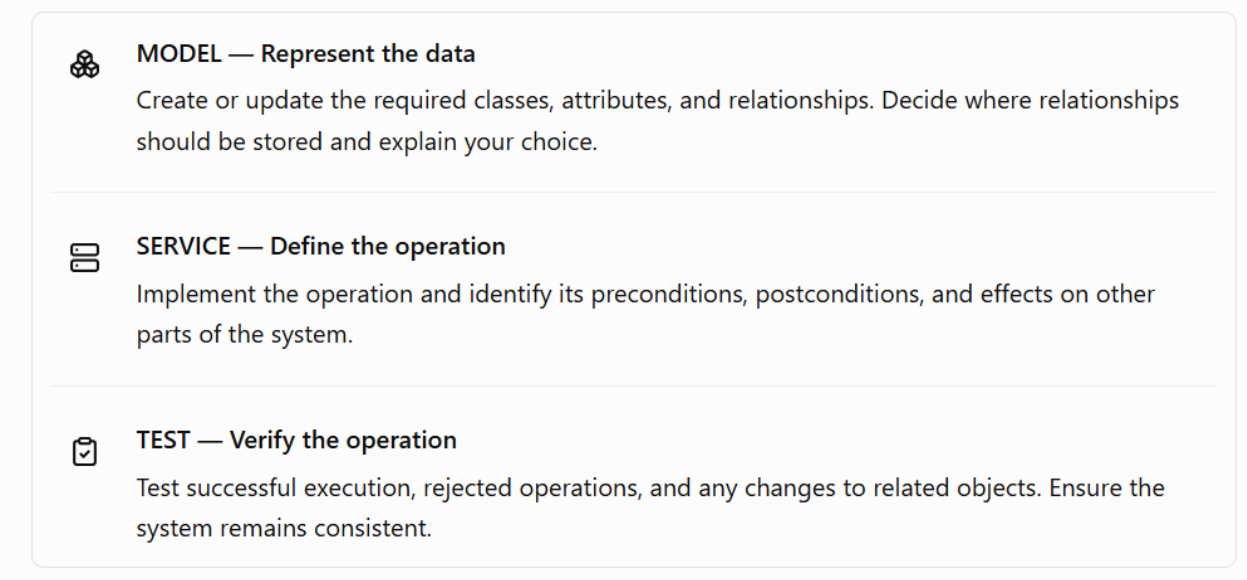
W3 PRACTICE

Layered Architecture

The goal of this **practice** is to provide an example of how to **structure** and **document** the **architecture** of **your own project**.





No AI tools allowed to solve this practice



How to submit?

- ✓ **Push** your final code on **your GitHub repository**
- ✓ Then **attach the GitHub path** to the MS Team assignment and **turn it in**

1. Business Context

The system is a simplified **Learning Management System (LMS)**.

The LMS manages:

- students
- courses
- course scores

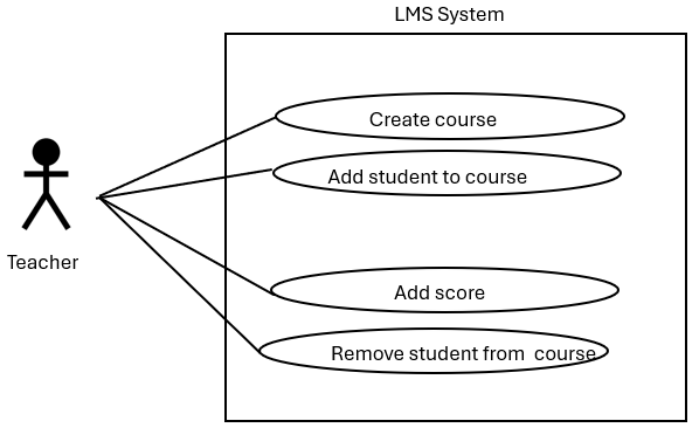
The system must allow an application to manage courses and students and record student results.

The application does **not** require a graphical user interface for this project.

The main objective is to design and implement a simple **Object-Oriented layered architecture**.

2. Functional requirements

The system can be represented with the following use cases:



FR1 — Create a course

The system must allow a new course to be created.

Example:

```
Create the course “Mobile Development”.
```

A course has:

- an ID
- a name
- a list of enrolled student IDs

FR2 — Add a student to a course

The system must allow an existing student to be enrolled in a course.

Example:

```
Add student S01 to the Mobile Development course.
```

A student must exist in the LMS before they can be enrolled in a course.

FR4 — Add a score for a student and a course

The system must allow a score to be recorded for a student and a course.

```
Student S01 receives 85 for the course  “Mobile Development”.
```

A score can only be added if:

- the student exists

- the course exists
- the student is enrolled in the course

FR5 — Remove a student

The system must allow a student to be removed from the LMS.

Removing a student is a **business operation**, not simply deleting a Student object.

When a student is removed:

1. the student is removed from the global student list
2. the student is removed from every course in which they are enrolled
3. all results belonging to this student are removed

```
Remove student S01.
```

After the operation, there must be no reference to S01 in the LMS.

3. Project Architecture and layer rules

The project uses three layers:

```
│
├── model/
│   ├── course.dart
│   ├── student.dart
│   └── course_result.dart
│
├── service/
│   └── lms_system.dart
│
└── test/
    └── lms_system_test.dart
```

Model layer

The **Model layer** contains the classes representing the business data.

It contains:

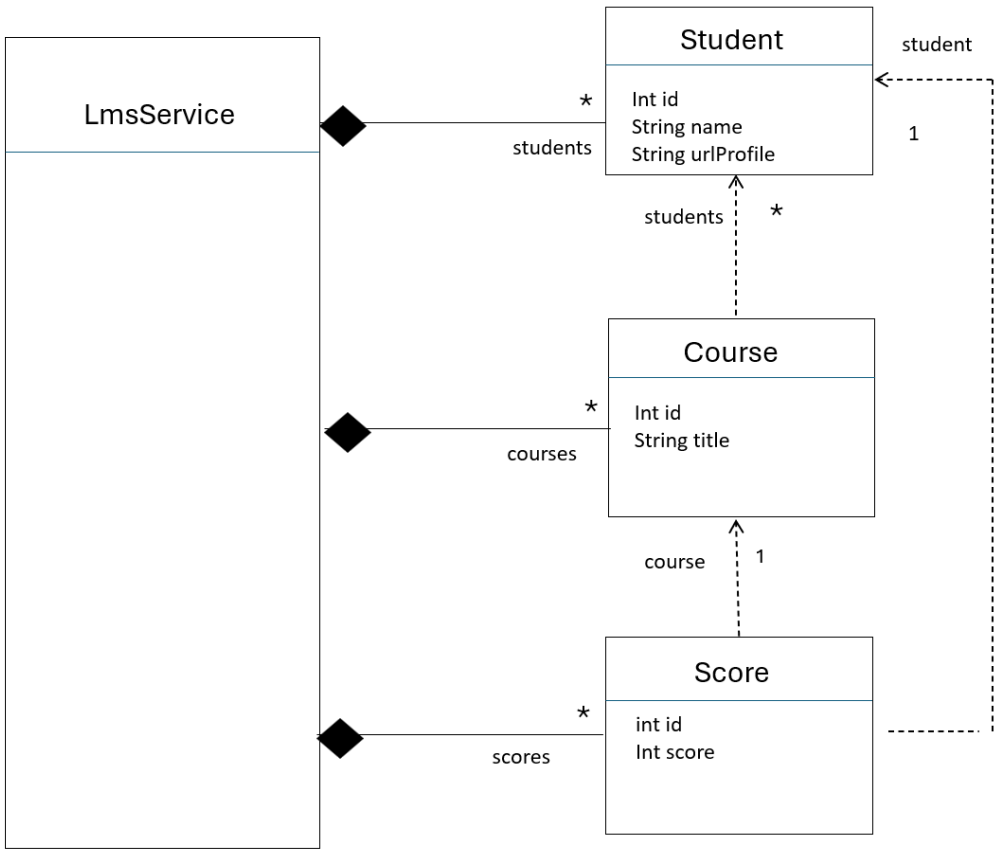
- `Student`
- `Course`
- `CourseResult`

The model classes **should mainly represent data**.

They should not contain the complete LMS business logic.

✓ The UML diagram represents the service classes and model classes.

✓ We should see the **relationship** between each class of the model



Service layer

✓ The service (LmsService) **gathers the application's data**.

```
LmsSystem
 ├── students
 ├── courses
 └── results
```

The **service** therefore **represents the current state of the LMS.**

✓ The service provides the **business operations** used by the rest of the application.

For example:

```
createCourse(...)
addStudentToCourse(...)
addAssignmentToCourse(...)
addScore(...)
removeStudent(...)
```

✓ The service is **responsible for enforcing the business rules.**

For example, the following situation should never happen:

```
Student S01 does not exist
```

but

```
Course C01 contains S01
```

Similarly:

```
Student S01 does not exist
```

but

```
CourseResult contains S01
```

The service operations must prevent or correct these inconsistent states.

💡 *For this simplified project, there is deliberately **no repository layer and no database**.*

Test layer

The **Test layer** verifies that the service behaves correctly.

Tests should:

```
Create LMS Service
      ↓
Add data
      ↓
Call LMS  service operation
      ↓
Check result
```

The tests should primarily test the **service API**, rather than directly manipulating the internal lists.

Scenario

1. Create two students
2. Create a course
3. Enroll both students
4. Add assignments
5. Add results for both students
6. Remove one student

Expected result

The removed student:

- no longer exists in `students`
- is no longer enrolled in the course
- has no results

The other student's data must remain unchanged.

4. What you need to do

Task 1 — Add and remove courses MANDATORY

MODEL

- Create a Course class with id and name.

SERVICE

Implement:

```
addCourse(coursed, courseName)
removeCourse(courseId)
```

Required preconditions:

❓ *What should happen if a course is removed while students are enrolled in it or have the results been recorded?*

TEST

```
Add and remove a course.
Reject a duplicate course ID.
Reject removal of a nonexistent course.
```

Task 2 — Enroll a student in a course MANDATORY

MODEL

- ✓ Represent the relationship between Student and Course.
- ✓ A student may enroll in several courses, and a course may contain several students.

SERVICE

```
- enrollStudent(studentId, courseId)
```

Required preconditions:

- ✓ The student exists.
- ✓ The course exists.
- ✓ The student is not already enrolled in the course.

TEST

```
Enroll an existing student in an existing course.
Verify that the enrollment is recorded.
Reject enrollment if the student does not exist.
Reject enrollment if the course does not exist.
Reject duplicate enrollment.
```

Task 3 — Remove a student from a course MANDATORY

```
- discardStudentFromCourse(studentId, courseId)
```

Required preconditions:

- ✓ What must be true before a student can be removed from a course?
- ✓ Should both the student and the course have to exist?
- ✓ What if the student is not enrolled in the course?
- ✓ What happens to the student's results in that course?

Important: Revisit removeStudent(studentId)

⚠️ If a student is removed from the LMS, how should their enrollments and results be handled?

Update the service operation **to preserve system consistency**.

TEST

- ✓ Remove an existing enrollment.
- ✓ Reject removal of a nonexistent enrollment.
- ✓ Verify that the student and course still exist after the enrollment is removed.
- ✓ Verify the chosen policy for associated results.
- ✓ Test removing a student who has enrollments in multiple courses.

Task 4 — Add a result to a course MANDATORY

MODEL

Create a **Result** class or propose an alternative design.

SERVICE

```
- addResult(studentId, courseId, score)
```

Required preconditions:

- ✓ Must the student exist?
- ✓ Must the course exist?
- ✓ Must the student be enrolled in the course?
- ✓ What is the valid score range?
- ✓ Can a student have two results for the same course?
- ✓ If multiple results are allowed, how will each result be identified?
- ✓ Choose and document the rules before implementation.

TEST

- ✓ Add a valid result.
- ✓ Reject a result for a nonexistent student or course.
- ✓ Reject a result for a student who is not enrolled, if enrollment is required.
- ✓ Reject an invalid score.

- ✓ Test the chosen policy for duplicate results.

Task 5 — Remove a student result MANDATORY

SERVICE

```
- removeResult(resultId), or an alternative signature justified by your model.
```

Required preconditions:

- ✓ Must the result exist?
- ✓ Who is allowed to remove it? For this practice, focus on data consistency rather than user permissions.
- ✓ Can a result be removed if the student is no longer enrolled?
- ✓ Does removing a result affect the student or course objects themselves?

TEST

- ✓ Remove an existing result.
- ✓ Reject removal of a nonexistent result.
- ✓ Verify that the result is no longer available.
- ✓ Verify that the student, course, and unrelated results remain unchanged.

5. Bonus tasks

Bonus A — **Introduce Assignments**

- ✓ Extend the model with Assignment:
  - a course contains multiple assignments,
  - student results are associated with assignments.

- ✓ Define the relationships, preconditions, and removal rules, then test them.

Bonus B — **Manage Assignment Stages**

- ✓ Add three stages: DRAFT, PUBLISHED, and CLOSED.
- ✓ Define the operations and valid stage transitions
  - including when results can be added or modified.
- ✓ Implement and test the rules.